

Wprowadzenie do Pandas

Przewodnik wyjaśnia kluczowe operacje na danych występujące w analizie danych i uczeniu maszynowym.

Krzysztof Chrzan

Wrocław 2026

krzysztof.chrzan@wroclaw.merito.pl

Podstawowe metody DataFrame

W tej sekcji omówimy podstawowe metody i atrybuty obiektów DataFrame w bibliotece Pandas, które są niezwykle przydatne do szybkiej analizy i zrozumienia struktury danych.

DataFrame to po prostu typ danych analogiczny do arkusza w Exelu, gdzie możemy mieć dane różnego typu jak liczby i napisy. DataFrames są użyteczne do przechowywania i przetwarzania danych w uczeniu maszynowym.

Metoda .head()

Wyjaśnienie:

Metoda `.head()` służy do wyświetlania pierwszych wierszy obiektu DataFrame. Domyślnie pokazuje 5 pierwszych wierszy, ale możesz określić inną liczbę, podając argument, np. `.head(3)`. Jest to bardzo przydatne do szybkiego podglądu struktury i zawartości danych bez konieczności wyświetlania całego zbioru. Analogiczne metoda `.tail()` pokazuje ostatnie wiersze obiektu DataFrame.

Przykład:

Utwórzmy prosty DataFrame zawierający dane studentów:

```
import pandas as pd

data = {
    'Imię': ['Anna', 'Piotr', 'Kasia', 'Jan', 'Ewa'],
    'Wiek': [22, 23, 21, 24, 22],
    'Ocena': [4.5, 3.8, 5.0, 4.2, None],
    'Przedmiot': ['Matematyka', 'Fizyka', 'Chemia', 'Biologia', 'Matematyka']
}
df = pd.DataFrame(data)

print(df.head(3))
```

Rzeczywisty OUTPUT:

	Imię	Wiek	Ocena	Przedmiot
0	Anna	22	4.5	Matematyka
1	Piotr	23	3.8	Fizyka
2	Kasia	21	5.0	Chemia

Metoda .info()

Wyjaśnienie:

Metoda `.info()` wyświetla zwięzłe podsumowanie obiektu DataFrame. Obejmuje ono informacje takie jak liczba wierszy, liczba kolumn, nazwy kolumn, typy danych (Dtype) w każdej kolumnie oraz liczbę wartości niepustych (Non-Null Count). Pomaga to szybko zrozumieć ogólną strukturę danych i zidentyfikować ewentualne brakujące wartości.

Przykład:

Użyjmy tego samego DataFrame z danymi studentów:

```
import pandas as pd

data = {
    'Imię': ['Anna', 'Piotr', 'Kasia', 'Jan', 'Ewa'],
    'Wiek': [22, 23, 21, 24, 22],
    'Ocena': [4.5, 3.8, 5.0, 4.2, None],
    'Przedmiot': ['Matematyka', 'Fizyka', 'Chemia', 'Biologia', 'Matematyka']
}
df = pd.DataFrame(data)

df.info()
```

Rzeczywisty OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0  Imię        5 non-null     object
1  Wiek        5 non-null     int64
2  Ocena       4 non-null     float64
3  Przedmiot   5 non-null     object
dtypes: float64(1), int64(1), object(2)
memory usage: 288.0+ bytes
```

Metoda .describe()

Wyjaśnienie:

Metoda `.describe()` generuje statystyki opisowe dla kolumn numerycznych w DataFrame. Obejmuje to takie miary jak liczba obserwacji (count), średnia (mean), odchylenie standardowe (std), wartość minimalna (min), wartość maksymalna (max) oraz kwartyle (25%, 50% - mediana, 75%). Jest to kluczowe narzędzie do szybkiego zrozumienia rozkładu danych numerycznych.

Przykład:

Użyjmy tego samego DataFrame z danymi studentów:

```
import pandas as pd

data = {
    'Imię': ['Anna', 'Piotr', 'Kasia', 'Jan', 'Ewa'],
    'Wiek': [22, 23, 21, 24, 22],
    'Ocena': [4.5, 3.8, 5.0, 4.2, None],
    'Przedmiot': ['Matematyka', 'Fizyka', 'Chemia', 'Biologia', 'Matematyka']
}
df = pd.DataFrame(data)

print(df.describe())
```

Rzeczywisty OUTPUT:

	Wiek	Ocena
count	5.000000	4.000000
mean	22.400000	4.375000
std	1.140175	0.505799
min	21.000000	3.800000
25%	22.000000	4.100000
50%	22.000000	4.350000
75%	23.000000	4.625000
max	24.000000	5.000000

Atrybut .shape

Wyjaśnienie:

Atrybut `.shape` zwraca krotkę (tuple) zawierającą wymiary DataFrame, czyli liczbę wierszy i liczbę kolumn. Format krotki to (liczba_wierszy, liczba_kolumn). Ważne jest, że `.shape` to atrybut, a nie metoda, więc nie używa się nawiasów przy jego wywoływaniu.

Przykład:

Użyjmy tego samego DataFrame z danymi studentów:

```
import pandas as pd

data = {
    'Imię': ['Anna', 'Piotr', 'Kasia', 'Jan', 'Ewa'],
    'Wiek': [22, 23, 21, 24, 22],
    'Ocena': [4.5, 3.8, 5.0, 4.2, None],
    'Przedmiot': ['Matematyka', 'Fizyka', 'Chemia', 'Biologia', 'Matematyka']
}
df = pd.DataFrame(data)

print(df.shape)
print(f"Liczba wierszy: {df.shape[0]}")
print(f"Liczba kolumn: {df.shape[1]}")
```

Rzeczywisty OUTPUT:

```
(5, 4)
Liczba wierszy: 5
Liczba kolumn: 4
```

Obsługa brakujących danych

Brakujące dane (często reprezentowane jako NaN - Not a Number) są powszechnym problemem w rzeczywistych zbiorach danych. Poniżej przedstawiono kluczowe metody biblioteki Pandas do identyfikacji i zarządzania nimi.

Metoda .isna()

Wyjaśnienie:

Metoda `.isna()` (lub `.isnull()`) jest używana do identyfikowania brakujących wartości w `DataFrame`. Zwraca ona `DataFrame` tej samej wielkości, zawierający wartości logiczne `True` w miejscach, gdzie dane są brakujące (NaN), oraz `False` tam, gdzie dane są kompletne. W połączeniu z metodą `.sum()` pozwala szybko policzyć liczbę brakujących wartości w każdej kolumnie.

Przykład Input (`DataFrame`):

```
import pandas as pd
import numpy as np

data = {
    'ID': [1, 2, 3, 4, 5],
    'Imię': ['Anna', 'Piotr', 'Kasia', 'Jan', 'Ewa'],
    'Ocena': [4.5, np.nan, 5.0, 3.8, np.nan],
    'Przedmiot': ['Matematyka', 'Fizyka', np.nan, 'Biologia', 'Chemia']
}
df = pd.DataFrame(data)

print(df)
```

	ID	Imię	Ocena	Przedmiot
0	1	Anna	4.5	Matematyka
1	2	Piotr	NaN	Fizyka
2	3	Kasia	5.0	NaN
3	4	Jan	3.8	Biologia
4	5	Ewa	NaN	Chemia

Zastosowanie metody:

```
print(df.isna().sum())
```

Rzeczywisty OUTPUT:

```
ID      0
Imię     0
Ocena    2
Przedmiot 1
dtype: int64
```

Metoda .fillna()

Wyjaśnienie:

Metoda `.fillna()` służy do uzupełniania brakujących wartości (NaN) w `DataFrame` lub `Series`. Można uzupełnić je np. średnią, medianą, lub najczęstszą wartością kolumny.

Przykład Input (`DataFrame`):

```
import pandas as pd
import numpy as np

data = {
    'ID': [1, 2, 3, 4, 5],
    'Imię': ['Anna', 'Piotr', 'Kasia', 'Jan', 'Ewa'],
    'Ocena': [4.5, np.nan, 5.0, 3.8, np.nan],
    'Przedmiot': ['Matematyka', 'Fizyka', np.nan, 'Biologia', 'Chemia']
}
df = pd.DataFrame(data)

print(df)
```

	ID	Imię	Ocena	Przedmiot
0	1	Anna	4.5	Matematyka
1	2	Piotr	NaN	Fizyka
2	3	Kasia	5.0	NaN
3	4	Jan	3.8	Biologia
4	5	Ewa	NaN	Chemia

Zastosowanie metody:

```
# Uzupełnianie brakujących ocen średnią wartością z kolumny 'Ocena'
df['Ocena'] = df['Ocena'].fillna(df['Ocena'].mean())
# Uzupełnianie brakujących przedmiotów wartością 'Brak_Danych'
df['Przedmiot'] = df['Przedmiot'].fillna('Brak_Danych')

print(df)
```

Rzeczywisty OUTPUT:

	ID	Imię	Ocena	Przedmiot
0	1	Anna	4.50	Matematyka
1	2	Piotr	4.43	Fizyka
2	3	Kasia	5.00	Brak_Danych
3	4	Jan	3.80	Biologia
4	5	Ewa	4.43	Chemia

Metoda .dropna()

Wyjaśnienie:

Metoda `.dropna()` służy do usuwania wierszy lub kolumn, które zawierają brakujące wartości (NaN). Jest to przydatne, gdy nie chcemy uzupełniać braków, a raczej pozbyć się niekompletnych obserwacji. Domyślnie usuwa wiersze zawierające choć jedną brakującą wartość.

Przykład Input (`DataFrame`):

```
import pandas as pd
import numpy as np

data = {
    'ID': [1, 2, 3, 4, 5],
    'Imię': ['Anna', 'Piotr', 'Kasia', 'Jan', 'Ewa'],
    'Ocena': [4.5, np.nan, 5.0, 3.8, np.nan],
    'Przedmiot': ['Matematyka', 'Fizyka', np.nan, 'Biologia', 'Chemia']
}
df = pd.DataFrame(data)

print(df)
```

	ID	Imię	Ocena	Przedmiot
0	1	Anna	4.5	Matematyka
1	2	Piotr	NaN	Fizyka
2	3	Kasia	5.0	NaN
3	4	Jan	3.8	Biologia
4	5	Ewa	NaN	Chemia

Zastosowanie metody:

```
# Usunięcie wszystkich wierszy, które zawierają brakujące wartości
df_cleaned = df.dropna()

print(df_cleaned)
```

Rzeczywisty OUTPUT:

	ID	Imię	Ocena	Przedmiot
0	1	Anna	4.5	Matematyka
3	4	Jan	3.8	Biologia

Analiza i grupowanie danych

Poniżej przedstawiamy kluczowe metody biblioteki Pandas do analizy i grupowania danych, takie jak grupowanie, liczenie unikalnych wartości oraz zaawansowane agregacje.

Metoda .groupby() - Grupowanie danych po kolumnie

Wyjaśnienie:

Metoda `.groupby()` służy do grupowania wierszy obiektu *DataFrame* na podstawie wartości jednej lub wielu kolumn. Po pogrupowaniu danych możemy wykonywać operacje agregujące (np. sumowanie, uśrednianie, liczenie) **oddzielnie dla każdej grupy**.

Jest to jedno z podstawowych narzędzi analizy danych. Pozwala na **numeryczne porównywanie grup**, np. porównanie liczby mieszkańców według kontynentów.

`groupby` jest także pomocne przy **wyborze zmiennych do modelu uczenia maszynowego**. Przykładowo, w modelu przewidującym cenę samochodu możemy porównać średnią cenę aut według kraju produkcji. Jeżeli średnie ceny istotnie różnią się między krajami, taka zmienna może być dobrym kandydatem do wykorzystania w modelu.

Przykładowy DataFrame INPUT:

```
import pandas as pd
sprzedaz = pd.DataFrame({
    'Region': ['Północ', 'Południe', 'Północ', 'Zachód', 'Południe'],
    'Kwota': [100, 150, 120, 200, 80]
})
print(sprzedaz)
```

Rzeczywisty OUTPUT:

	Region	Kwota
0	Północ	100
1	Południe	150
2	Północ	120
3	Zachód	200
4	Południe	80

Prosty kod z użyciem `.groupby()`:

```
# Grupowanie po kolumnie 'Region' i sumowanie 'Kwota' dla każdego regionu
print(sprzedaz.groupby('Region')['Kwota'].sum())
```

Rzeczywisty OUTPUT:

	Region
Południe	230
Północ	220
Zachód	200
Name: Kwota, dtype: int64	

Zadanie dla studenta:

Dla poniższego DataFrame z danymi o produktach, pogrupuj dane według 'Kategoria' i oblicz średnią 'Cena' dla każdej kategorii.

```
produkty = pd.DataFrame({
    'Kategoria': ['Elektronika', 'Ubrania', 'Elektronika', 'Dom', 'Ubrania'],
    'Cena': [500, 150, 700, 50, 200]
})
```

Metoda .value_counts() - Liczenie unikalnych wartości

Wyjaśnienie:

Metoda `.value_counts()` jest używana do zliczania wystąpień unikalnych wartości w kolumnie Series (kolumnie DataFrame). Zwraca Series zawierające unikalne wartości jako indeks i ich liczbę wystąpień jako wartości, posortowane malejąco. Opcjonalny parametr `normalize=True` zwraca proporcje zamiast liczb bezwzględnych.

Przykładowy DataFrame INPUT:

```
import pandas as pd
zamowienia = pd.DataFrame({
    'ID': [1, 2, 3, 4, 5, 6],
    'Status': ['Nowe', 'Wysłane', 'Nowe', 'Anulowane', 'Wysłane', 'Nowe']
})
print(zamowienia)
```

Rzeczywisty OUTPUT:

	ID	Status
0	1	Nowe
1	2	Wysłane
2	3	Nowe
3	4	Anulowane
4	5	Wysłane
5	6	Nowe

Prosty kod z użyciem `.value_counts()`:

```
# Liczenie wystąpień każdego statusu
print(zamowienia['Status'].value_counts())
```

Rzeczywisty OUTPUT:

	Status
Nowe	3
Wysłane	2
Anulowane	1
Name: Status, dtype: int64	

Zadanie dla studenta:

Dla poniższego DataFrame z ocenami studentów, oblicz proporcje (udział procentowy) wystąpień każdej oceny w kolumnie 'Ocena'.

```
oceny = pd.DataFrame({
    'Student': ['Anna', 'Bartek', 'Anna', 'Celina', 'Bartek', 'Celina'],
    'Ocena': [4, 5, 4, 3, 5, 5]
})
```

Metoda .agg() - Agregacja danych (suma, średnia, liczba)

Wyjaśnienie:

Metoda `.agg()` pozwala na wykonywanie wielu operacji agregujących jednocześnie na jednej lub wielu kolumnach, często w połączeniu z `.groupby()`. Można przekazać pojedynczą funkcję agregującą (np. `'sum'`, `'mean'`) lub listę funkcji, a nawet słownik mapujący kolumny na funkcje, co daje dużą elastyczność w analizie.

Przykładowy DataFrame INPUT:

```
import pandas as pd
dane_firmy = pd.DataFrame({
    'Dział': ['HR', 'IT', 'HR', 'Sprzedaż', 'IT', 'Sprzedaż'],
    'Wynagrodzenie': [5000, 7000, 5500, 6000, 8000, 6200],
    'Staż_lata': [2, 5, 3, 4, 6, 3]
})
print(dane_firmy)
```

Rzeczywisty OUTPUT:

	Dział	Wynagrodzenie	Staż_lata
0	HR	5000	2
1	IT	7000	5
2	HR	5500	3
3	Sprzedaż	6000	4
4	IT	8000	6
5	Sprzedaż	6200	3

Prosty kod z użyciem `.agg()`:

```
# Agregacja wielu statystyk dla różnych kolumn po pogrupowaniu
raport = dane_firmy.groupby('Dział').agg(
    Srednie_Wynagrodzenie=('Wynagrodzenie', 'mean'),
    Liczba_Pracownikow=('Dział', 'size'),
    Max_Staz=('Staż_lata', 'max')
)
print(raport)
```

Rzeczywisty OUTPUT:

	Srednie_Wynagrodzenie	Liczba_Pracownikow	Max_Staz
Dział			
HR	5250.0	2	3
IT	7500.0	2	6
Sprzedaż	6100.0	2	4

Zadanie dla studenta:

Dla poniższego DataFrame z wynikami testów, pogrupuj dane według 'Przedmiot' i użyj `.agg()`, aby obliczyć zarówno średnią, jak i maksymalny wynik dla każdego przedmiotu w kolumnie 'Wynik'.

```
wyniki_testow = pd.DataFrame({
    'Przedmiot': ['Matematyka', 'Fizyka', 'Matematyka', 'Chemia', 'Fizyka'],
    'Wynik': [85, 92, 78, 90, 88]
})
```

Operacje na danych i duplikaty

Metoda .duplicated() - Sprawdzanie duplikatów

Wyjaśnienie:

Metoda .duplicated() służy do identyfikowania duplikatów w DataFrame lub Series. Zwraca Series wartości boolowskich (True/False), gdzie True wskazuje na wiersz, który jest duplikatem wcześniejszego wiersza (domyślnie keep='first'). Można ją zastosować do całego DataFrame lub do konkretnej kolumny/podzbioru kolumn.

Przykładowy DataFrame INPUT:

```
import pandas as pd
dane_zamowien = pd.DataFrame({
    'ID_Zamowienia': [1, 2, 3, 2, 4, 1],
    'Produkt': ['Laptop', 'Mysz', 'Klawiatura', 'Mysz', 'Monitor', 'Laptop'],
    'Ilosc': [1, 2, 1, 2, 1, 1]
})
print("Oryginalny DataFrame:")
print(dane_zamowien)
```

Rzeczywisty OUTPUT:

```
Oryginalny DataFrame:
  ID_Zamowienia  Produkt  Ilosc
0             1   Laptop     1
1             2     Mysz     2
2             3 Klawiatura     1
3             2     Mysz     2
4             4   Monitor     1
5             1   Laptop     1
```

Użycie .duplicated():

```
# Sprawdzenie, które wiersze są duplikatami
print("Wynik dane_zamowien.duplicated():")
print(dane_zamowien.duplicated())

# Wyświetlenie tylko wierszy będących duplikatami
print("\nWiersze będące duplikatami:")
print(dane_zamowien[dane_zamowien.duplicated()])
```

Rzeczywisty OUTPUT:

```
Wynik dane_zamowien.duplicated():
0    False
1    False
2    False
3     True
4    False
5     True
dtype: bool
```

```
Wiersze będące duplikatami:
  ID_Zamowienia  Produkt  Ilosc
3             2     Mysz     2
5             1   Laptop     1
```

Zadanie dla studenta:

Dla poniższego DataFrame z listą studentów, znajdź i wyświetl wszystkie wiersze, które są duplikatami (tj. imię, nazwisko i wiek są takie same).

```
studenci = pd.DataFrame({
    'Imie': ['Anna', 'Piotr', 'Anna', 'Katarzyna', 'Piotr'],
    'Nazwisko': ['Kowalska', 'Nowak', 'Kowalska', 'Wójcik', 'Nowak'],
    'Wiek': [20, 22, 20, 21, 22]
})
print("DataFrame studenci:")
print(studenci)
```

Metoda .drop_duplicates() - Usuwanie duplikatów

Wyjaśnienie:

Metoda .drop_duplicates() usuwa z DataFrame wiersze, które są identyczne. Domyślnie pozostawia pierwsze wystąpienie duplikatu (keep='first'). Możesz określić, które kolumny mają być brane pod uwagę przy wykrywaniu duplikatów za pomocą parametru subset oraz które wystąpienia duplikatów zostawić ('first', 'last' lub False, aby usunąć wszystkie).

Przykładowy DataFrame INPUT:

```
import pandas as pd
dane_zamowien_do_usuniecia = pd.DataFrame({
    'ID_Zamowienia': [1, 2, 3, 2, 4, 1],
    'Produkt': ['Laptop', 'Mysz', 'Klawiatura', 'Mysz', 'Monitor', 'Laptop'],
    'Ilosc': [1, 2, 1, 2, 1, 1]
})
print("Oryginalny DataFrame z duplikatami:")
print(dane_zamowien_do_usuniecia)
```

Rzeczywisty OUTPUT:

```
Oryginalny DataFrame z duplikatami:
  ID_Zamowienia  Produkt  Ilosc
0             1   Laptop     1
1             2     Mysz     2
2             3 Klawiatura     1
3             2     Mysz     2
4             4   Monitor     1
5             1   Laptop     1
```

Użycie .drop_duplicates():

```
# Usunięcie wszystkich duplikatów, pozostawiając pierwsze wystąpienie
dane_unikalne_po_id = dane_zamowien_do_usuniecia.drop_duplicates()
print("\nDataFrame po usunięciu duplikatów:")
print(dane_bez_duplikatow)

# Usunięcie duplikatów na podstawie kolumny 'ID_Zamowienia', pozostawiając ostatnie wystąpienie
dane_unikalne_po_id = dane_zamowien_do_usuniecia.drop_duplicates(subset=['ID_Zamowienia'], keep='last')
print("\nDataFrame po usunięciu duplikatów na podstawie ID_Zamowienia (keep='last'):")
print(dane_unikalne_po_id)
```

Rzeczywisty OUTPUT:

```
DataFrame po usunięciu duplikatów:
  ID_Zamowienia  Produkt  Ilosc
0             1   Laptop     1
1             2     Mysz     2
2             3 Klawiatura     1
4             4   Monitor     1

DataFrame po usunięciu duplikatów na podstawie ID_Zamowienia (keep='last'):
  ID_Zamowienia  Produkt  Ilosc
2             3 Klawiatura     1
3             2     Mysz     2
4             4   Monitor     1
5             1   Laptop     1
```

Zadanie dla studenta:

Dla poniższego DataFrame, usuń wszystkie duplikaty wierszy. Następnie, stwórz nowy DataFrame, w którym usuniesz duplikaty bazując tylko na kolumnie 'Email', zachowując pierwsze wystąpienie każdego unikalnego emaila.

```
uzytkownicy = pd.DataFrame({
    'UzytkownikID': [101, 102, 103, 102, 104, 101],
    'Email': ['a@example.com', 'b@example.com', 'c@example.com', 'b@example.com', 'd@example.com', 'a@example.com'],
    'Kraj': ['PL', 'DE', 'PL', 'DE', 'FR', 'PL']
})
print("Oryginalny DataFrame uzytkownicy:")
print(uzytkownicy)
```

Metoda .astype() - Zmiana typu danych kolumny

Wyjaśnienie:

Metoda .astype() służy do konwersji typu danych (dtype) jednej lub wielu kolumn w DataFrame. Jest to kluczowe, gdy dane są wczytywane w nieodpowiednim formacie (np. liczby jako stringi) lub gdy potrzebujemy konkretnego typu danych do obliczeń czy oszczędności pamięci. Najczęściej używane typy to int, float, str, datetime, category.

Przykładowy DataFrame INPUT:

```
import pandas as pd
dane_ceny = pd.DataFrame({
    'Nazwa': ['Produkt A', 'Produkt B', 'Produkt C'],
    'Cena_string': ['100', '150.50', '200'],
    'Dostepnosc_bool_int': [1, 0, 1]
})
print("Oryginalny DataFrame dane_ceny:")
print(dane_ceny)
print("\nTypy danych przed konwersją:")
print(dane_ceny.dtypes)
```

Rzeczywisty OUTPUT:

```
Oryginalny DataFrame dane_ceny:
  Nazwa  Cena_string  Dostepnosc_bool_int
0  Produkt A      100                  1
1  Produkt B   150.50                  0
2  Produkt C     200                  1
```

```
Typy danych przed konwersją:
Nazwa          object
Cena_string    object
Dostepnosc_bool_int  int64
dtype: object
```

Użycie .astype():

```
# Konwersja kolumny 'Cena_string' na typ float
dane_ceny['Cena_float'] = dane_ceny['Cena_string'].astype(float)

# Konwersja kolumny 'Dostepnosc_bool_int' na typ bool
dane_ceny['Dostepnosc_bool'] = dane_ceny['Dostepnosc_bool_int'].astype(bool)

print("\nDataFrame po konwersji typów:")
print(dane_ceny)
print("\nTypy danych po konwersji:")
print(dane_ceny.dtypes)
```

Rzeczywisty OUTPUT:

```
DataFrame po konwersji typów:
  Nazwa  Cena_string  Dostepnosc_bool_int  Cena_float  Dostepnosc_bool
0  Produkt A      100                  1     100.0         True
1  Produkt B   150.50                  0     150.5        False
2  Produkt C     200                  1     200.0         True

Typy danych po konwersji:
Nazwa          object
Cena_string    object
Dostepnosc_bool_int  int64
Cena_float      float64
Dostepnosc_bool    bool
dtype: object
```

Zadanie dla studenta:

Dla poniższego DataFrame z danymi o pracownikach, kolumna 'Lata_Doswiadczenia' została wczytana jako string. Zmierz typ danych tej kolumny na liczbę całkowitą (int). Następnie, kolumnę 'ID_Pracownika' również przekonwertuj na typ str, ponieważ może zawierać prefiks i jest traktowana jako identyfikator tekstowy.

```
pracownicy = pd.DataFrame({
    'ID_Pracownika': [1001, 1002, 1003],
    'Imie': ['Jan', 'Ewa', 'Adam'],
    'Lata_Doswiadczenia': ['5', '3', '7']
})
print("DataFrame pracownicy przed konwersją:")
print(pracownicy)
print("\nTypy danych przed konwersją:")
print(pracownicy.dtypes)
```

Indeksowanie i selekcja danych

Metoda .loc[] - Selekcja po etykietach

Wyjaśnienie:

Metoda .loc[] służy do selekcji danych w DataFrame na podstawie etykiet (nazw) wierszy i kolumn. Jest to potężne narzędzie, które pozwala na wybieranie pojedynczych elementów, zakresów etykiet (włącznie z końcową etykietą) oraz list etykiet. Format użycia to df.loc[etykiety_wierszy, etykiety_kolumn].

Przykładowy DataFrame INPUT:

```
import pandas as pd
studenci = pd.DataFrame({
    'ID': [101, 102, 103, 104, 105],
    'Imie': ['Anna', 'Piotr', 'Kasia', 'Marek', 'Ewa'],
    'Ocena': [4.5, 3.0, 5.0, 3.5, 4.0]
}, index=['s1', 's2', 's3', 's4', 's5']) # Indeks wierszy to etykiety
print("Oryginalny DataFrame studenci:")
print(studenci)
```

Rzeczywisty OUTPUT:

Oryginalny DataFrame studenci:

```
ID  Imie  Ocena
s1  101  Anna   4.5
s2  102  Piotr   3.0
s3  103  Kasia   5.0
s4  104  Marek   3.5
s5  105   Ewa   4.0
```

Użycie .loc[]:

```
# Wybór pojedynczego wiersza po etykiecie indeksu
print("Wybór wiersza 's3':")
print(studenci.loc['s3'])

# Wybór wielu wierszy po liście etykiet
print("\nWybór wierszy 's1', 's5':")
print(studenci.loc[['s1', 's5']])

# Wybór zakresu wierszy i kolumn po etykietach
print("\nWybór wierszy od 's2' do 's4' i kolumn od 'Imie' do 'Ocena':")
print(studenci.loc['s2':'s4', 'Imie':'Ocena'])

# Wybór wszystkich wierszy i pojedynczej kolumny
print("\nWybór wszystkich wierszy i kolumny 'Ocena':")
print(studenci.loc[:, 'Ocena'])
```

Rzeczywisty OUTPUT:

```
Wybór wiersza 's3':
ID    103
Imie   Kasia
Ocena   5.0
Name: s3, dtype: object

Wybór wierszy 's1', 's5':
ID  Imie  Ocena
s1  101  Anna   4.5
s5  105   Ewa   4.0

Wybór wierszy od 's2' do 's4' i kolumn od 'Imie' do 'Ocena':
Imie  Ocena
s2  Piotr   3.0
s3  Kasia   5.0
s4  Marek   3.5

Wybór wszystkich wierszy i kolumny 'Ocena':
s1    4.5
s2    3.0
s3    5.0
s4    3.5
s5    4.0
Name: Ocena, dtype: float64
```

Zadanie dla studenta:

Dla DataFrame studenci, użyj .loc[] aby wybrać wiersze o etykietach 's1' i 's4' oraz tylko kolumnę 'ID'.

Metoda .iloc[] - Selekcja po pozycji (indeksie numerycznym)

Wyjaśnienie:

Metoda .iloc[] służy do selekcji danych w DataFrame na podstawie numerycznych pozycji (indeksów) wierszy i kolumn. Działa analogicznie do cięcia list w Pythonie, co oznacza, że zakresy są wyłączone dla górnej granicy. Format użycia to df.iloc[pozycje_wierszy, pozycje_kolumn].

Przykładowy DataFrame INPUT:

```
import pandas as pd
studenci = pd.DataFrame({
    'ID': [101, 102, 103, 104, 105],
    'Imie': ['Anna', 'Piotr', 'Kasia', 'Marek', 'Ewa'],
    'Ocena': [4.5, 3.0, 5.0, 3.5, 4.0]
}, index=['s1', 's2', 's3', 's4', 's5'])
print("Oryginalny DataFrame studenci:")
print(studenci)
```

Rzeczywisty OUTPUT:

Oryginalny DataFrame studenci:

```
ID  Imie  Ocena
s1  101  Anna   4.5
s2  102  Piotr   3.0
s3  103  Kasia   5.0
s4  104  Marek   3.5
s5  105   Ewa   4.0
```

Użycie .iloc[]:

```
# Wybór pojedynczego wiersza po pozycji
print("Wybór wiersza o indeksie 2 (trzeci wiersz):")
print(studenci.iloc[2])

# Wybór wielu wierszy po liście pozycji
print("\nWybór wierszy o indeksach 0, 4:")
print(studenci.iloc[[0, 4]])

# Wybór zakresu wierszy i kolumn po pozycjach (górna granica wyłączona)
print("\nWybór wierszy od indeksu 1 do 4 (wiersze 1,2,3) i kolumn od indeksu 1 do 3 (kolumny 1,2):")
print(studenci.iloc[1:4, 1:3])

# Wybór wszystkich wierszy i kolumny o indeksie 2
print("\nWybór wszystkich wierszy i kolumny o indeksie 2 ('Ocena'):")
print(studenci.iloc[:, 2])
```

Rzeczywisty OUTPUT:

```
Wybór wiersza o indeksie 2 (trzeci wiersz):
ID    103
Imie   Kasia
Ocena   5.0
Name: s3, dtype: object

Wybór wielu wierszy o indeksach 0, 4:
ID  Imie  Ocena
s1  101  Anna   4.5
s5  105   Ewa   4.0

Wybór wierszy od indeksu 1 do 4 (wiersze 1,2,3) i kolumn od indeksu 1 do 3 (kolumny 1,2):
Imie  Ocena
s2  Piotr   3.0
s3  Kasia   5.0
s4  Marek   3.5

Wybór wszystkich wierszy i kolumny o indeksie 2 ('Ocena'):
s1    4.5
s2    3.0
s3    5.0
s4    3.5
s5    4.0
Name: Ocena, dtype: float64
```

Zadanie dla studenta:

Dla DataFrame studenci, użyj .iloc[] aby wybrać wiersze o pozycjach 0, 1 i 3 oraz kolumny o pozycjach 0 i 1.

Metoda .set_index() - Ustawienie kolumny jako indeks

Wyjaśnienie:

Metoda .set_index() pozwala na ustawienie jednej lub więcej istniejących kolumn DataFrame jako nowego indeksu (etykiety wierszy). Jest to przydatne, gdy chcemy, aby wiersze były identyfikowane unikalnymi wartościami z konkretnej kolumny, zamiast domyślnych indeksów numerycznych.

Przykładowy DataFrame INPUT:

```
import pandas as pd
produkty = pd.DataFrame({
    'ProduktID': [1, 2, 3, 4],
    'Nazwa': ['Laptop', 'Mysz', 'Klawiatura', 'Monitor'],
    'Cena': [2500, 150, 300, 1200]
})
print("Oryginalny DataFrame produkty:")
print(produkty)
print("\nTyp indeksu przed zmianą:", produkty.index.dtype)
```

Rzeczywisty OUTPUT:

Oryginalny DataFrame produkty:

```
ProduktID  Nazwa  Cena
0         1  Laptop  2500
1         2   Mysz   150
2         3 Klawiatura  300
3         4  Monitor 1200
```

Typ indeksu przed zmianą: int64

Użycie .set_index():

```
# Ustawienie kolumny 'ProduktID' jako indeksu
produkty_z_indeksem = produkty.set_index('ProduktID')
print("\nDataFrame produkty po ustawieniu 'ProduktID' jako indeksu:")
print(produkty_z_indeksem)
print("\nTyp indeksu po zmianie:", produkty_z_indeksem.index.dtype)
```

Rzeczywisty OUTPUT:

DataFrame produkty po ustawieniu 'ProduktID' jako indeksu:

```
Nazwa  Cena
ProduktID
1      Laptop  2500
2        Mysz   150
3  Klawiatura  300
4    Monitor 1200
```

Typ indeksu po zmianie: int64

Zadanie dla studenta:

Dla poniższego DataFrame, ustaw kolumnę 'Miasto' jako indeks.

```
dane_pogodowe = pd.DataFrame({
    'Miasto': ['Warszawa', 'Kraków', 'Gdańsk'],
    'Temperatura': [20, 18, 15],
    'Opady': [True, False, True]
})
print("Oryginalny DataFrame dane_pogodowe:")
print(dane_pogodowe)
```

Filtrowanie warunkowe

Wyjaśnienie:

Filtrowanie warunkowe pozwala na wybór wierszy z DataFrame, które spełniają określone kryteria. Tworzymy serię wartości boolowskich (True/False) na podstawie warunku logicznego, a następnie używamy jej do indeksowania DataFrame. Wiersze, dla których warunek jest prawdziwy, zostaną zachowane.

Przykładowy DataFrame INPUT:

```
import pandas as pd
zamowienia = pd.DataFrame({
    'ID_Klienta': [1, 2, 1, 3, 2],
    'Kwota': [150.00, 200.50, 75.20, 300.00, 120.00],
    'Status': ['Zrealizowane', 'W toku', 'Anulowane', 'Zrealizowane', 'W toku']
})
print("Oryginalny DataFrame zamowienia:")
print(zamowienia)
```

Rzeczywisty OUTPUT:

Oryginalny DataFrame zamowienia:

```
ID_Klienta  Kwota  Status
0         1  150.00 Zrealizowane
1         2  200.50   W toku
2         1   75.20 Anulowane
3         3  300.00 Zrealizowane
4         2  120.00   W toku
```

Użycie filtrowania warunkowego:

```
# Filtrowanie wierszy, gdzie 'Kwota' jest większa niż 100
print("Zamówienia o kwocie większej niż 100:")
print(zamowienia[zamowienia['Kwota'] > 100])

# Filtrowanie wierszy, gdzie 'Status' to 'Zrealizowane'
print("\nZamówienia ze statusem 'Zrealizowane':")
print(zamowienia[zamowienia['Status'] == 'Zrealizowane'])

# Filtrowanie z użyciem wielu warunków (np. 'Kwota' > 100 ORAZ 'Status' == 'W toku')
print("\nZamówienia o kwocie > 100 i statusie 'W toku':")
print(zamowienia[(zamowienia['Kwota'] > 100) & (zamowienia['Status'] == 'W toku')])
```

Rzeczywisty OUTPUT:

Zamówienia o kwocie większej niż 100:

```
ID_Klienta  Kwota  Status
0         1  150.00 Zrealizowane
1         2  200.50   W toku
3         3  300.00 Zrealizowane
4         2  120.00   W toku
```

Zamówienia ze statusem 'Zrealizowane':

```
ID_Klienta  Kwota  Status
0         1  150.00 Zrealizowane
3         3  300.00 Zrealizowane
```

Zamówienia o kwocie > 100 i statusie 'W toku':

```
ID_Klienta  Kwota  Status
1         2  200.50   W toku
4         2  120.00   W toku
```

Zadanie dla studenta:

Dla DataFrame zamowienia, znajdź wszystkie zamówienia, których ID_Klienta jest równe 1, a Status nie jest 'Anulowane'.

```
# Napisz kod tutaj:
```

Łączenie danych i tworzenie lub usuwanie nowych kolumn

Metoda .drop() - Usuwanie kolumn/wierszy

Wyjaśnienie:

Metoda `.drop()` służy do usuwania określonych wierszy lub kolumn z `DataFrame`. Aby usunąć kolumnę, należy podać jej nazwę oraz `axis=1` (lub `axis='columns'`). Do usunięcia wiersza służy podanie etykiety wiersza oraz `axis=0` (lub `axis='index'`). Domyślnie `.drop()` zwraca nowy `DataFrame`, nie modyfikując oryginalnego; aby zmodyfikować oryginalny `DataFrame`, należy ustawić `inplace=True`.

Przykładowy DataFrame INPUT:

```
import pandas as pd
dane_firmy = pd.DataFrame({
    'PracownikID': [1, 2, 3, 4],
    'Imie': ['Alicja', 'Bartek', 'Cecylia', 'Dariusz'],
    'Dzial': ['IT', 'HR', 'IT', 'Marketing'],
    'Pensja': [7000, 5000, 7500, 6000]
})
print("Oryginalny DataFrame dane_firmy:")
print(dane_firmy)
```

Rzeczywisty OUTPUT:

Oryginalny DataFrame dane_firmy:

	PracownikID	Imie	Dzial	Pensja
0	1	Alicja	IT	7000
1	2	Bartek	HR	5000
2	3	Cecylia	IT	7500
3	4	Dariusz	Marketing	6000

Użycie .drop():

```
# Usunięcie kolumny 'PracownikID'
dane_bez_id = dane_firmy.drop('PracownikID', axis=1)
print("\nDataFrame po usunięciu kolumny 'PracownikID':")
print(dane_bez_id)

# Usunięcie wiersza o indeksie 1 (drugiego wiersza)
dane_bez_wiersza = dane_firmy.drop(1, axis=0)
print("\nDataFrame po usunięciu wiersza o indeksie 1:")
print(dane_bez_wiersza)
```

Rzeczywisty OUTPUT:

DataFrame po usunięciu kolumny 'PracownikID':

	Imie	Dzial	Pensja
0	Alicja	IT	7000
1	Bartek	HR	5000
2	Cecylia	IT	7500
3	Dariusz	Marketing	6000

DataFrame po usunięciu wiersza o indeksie 1:

	PracownikID	Imie	Dzial	Pensja
0	1	Alicja	IT	7000
2	3	Cecylia	IT	7500
3	4	Dariusz	Marketing	6000

Zadanie dla studenta:

Dla DataFrame `dane_firmy`, utwórz nowy DataFrame, usuwając kolumnę 'Pensja'.

Tworzenie nowych kolumn

Wyjaśnienie:

Możemy tworzyć nowe kolumny w `DataFrame`, przypisując wartości do nieistniejącej jeszcze nazwy kolumny. Wartości te mogą być stałe, wynikami operacji na istniejących kolumnach lub seriami danych. Jest to często używane do inżynierii cech (feature engineering).

Przykładowy DataFrame INPUT:

```
import pandas as pd
zamowienia = pd.DataFrame({
    'ID_Produktu': [101, 102, 103, 104],
    'Cena_Jednostkowa': [25.50, 10.00, 50.75, 12.20],
    'Ilosc': [2, 5, 1, 3]
})
print("Oryginalny DataFrame zamowienia:")
print(zamowienia)
```

Rzeczywisty OUTPUT:

Oryginalny DataFrame zamowienia:

	ID_Produktu	Cena_Jednostkowa	Ilosc
0	101	25.50	2
1	102	10.00	5
2	103	50.75	1
3	104	12.20	3

Użycie tworzenia nowych kolumn:

```
# Tworzenie kolumny 'Wartosc_Laczna' na podstawie innych kolumn
zamowienia['Wartosc_Laczna'] = zamowienia['Cena_Jednostkowa'] * zamowienia['Ilosc']
print("\nDataFrame z nową kolumną 'Wartosc_Laczna':")
print(zamowienia)

# Tworzenie kolumny ze stałą wartością
zamowienia['Waluta'] = 'PLN'
print("\nDataFrame z nową kolumną 'Waluta':")
print(zamowienia)

# Tworzenie kolumny z wartościami logicznymi na podstawie warunku
zamowienia['Duze_Zamowienie'] = zamowienia['Wartosc_Laczna'] > 100
print("\nDataFrame z nową kolumną 'Duze_Zamowienie':")
print(zamowienia)
```

Rzeczywisty OUTPUT:

DataFrame z nową kolumną 'Wartosc_Laczna':

	ID_Produktu	Cena_Jednostkowa	Ilosc	Wartosc_Laczna
0	101	25.50	2	51.00
1	102	10.00	5	50.00
2	103	50.75	1	50.75
3	104	12.20	3	36.60

DataFrame z nową kolumną 'Waluta':

	ID_Produktu	Cena_Jednostkowa	Ilosc	Wartosc_Laczna	Waluta
0	101	25.50	2	51.00	PLN
1	102	10.00	5	50.00	PLN
2	103	50.75	1	50.75	PLN
3	104	12.20	3	36.60	PLN

DataFrame z nową kolumną 'Duze_Zamowienie':

	ID_Produktu	Cena_Jednostkowa	Ilosc	Wartosc_Laczna	Waluta	Duze_Zamowienie
0	101	25.50	2	51.00	PLN	False
1	102	10.00	5	50.00	PLN	False
2	103	50.75	1	50.75	PLN	False
3	104	12.20	3	36.60	PLN	False

Zadanie dla studenta:

Dla DataFrame `zamowienia`, utwórz nową kolumnę 'Rabat', która będzie wynosić 0.10 (10%) dla zamówień, których 'Wartosc_Laczna' jest większa niż 40, w przeciwnym razie 0.00.

Funkcja pd.concat() - Łączenie DataFramów

Wyjaśnienie:

Funkcja `pd.concat()` służy do łączenia obiektów Pandas (`DataFrame`ów lub `Series`) wzdłuż określonej osi. Domyślnie łączy wierszami (`axis=0`), czyli dokłada jeden `DataFrame` pod drugim. Można również łączyć kolumnami (`axis=1`), co wymaga zgodności indeksów wierszy.

Przykładowy DataFrame INPUT:

```
import pandas as pd
df1 = pd.DataFrame({
    'Imie': ['Anna', 'Piotr'],
    'Wiek': [28, 34]
})
df2 = pd.DataFrame({
    'Imie': ['Kasia', 'Marek'],
    'Wiek': [22, 45]
})
print("Pierwszy DataFrame (df1):")
print(df1)
print("\nDrugi DataFrame (df2):")
print(df2)

df3 = pd.DataFrame({
    'Miasto': ['Warszawa', 'Kraków'],
    'Kraj': ['Polska', 'Polska']
})
print("\nTrzeci DataFrame (df3):")
print(df3)
```

Rzeczywisty OUTPUT:

Pierwszy DataFrame (df1):

	Imie	Wiek
0	Anna	28
1	Piotr	34

Drugi DataFrame (df2):

	Imie	Wiek
0	Kasia	22
1	Marek	45

Trzeci DataFrame (df3):

	Miasto	Kraj
0	Warszawa	Polska
1	Kraków	Polska

Użycie pd.concat():

```
# Łączenie df1 i df2 wierszami (axis=0)
df_wiersze = pd.concat([df1, df2])
print("\nPołączone DataFrame'y wierszami:")
print(df_wiersze)

# Łączenie df1 i df3 kolumnami (axis=1) - wymagana zgodność indeksów
df_kolumny = pd.concat([df1, df3], axis=1)
print("\nPołączone DataFrame'y kolumnami:")
print(df_kolumny)
```

Rzeczywisty OUTPUT:

Połączone DataFrame'y wierszami:

	Imie	Wiek
0	Anna	28
1	Piotr	34
0	Kasia	22
1	Marek	45

Połączone DataFrame'y kolumnami:

	Imie	Wiek	Miasto	Kraj
0	Anna	28	Warszawa	Polska
1	Piotr	34	Kraków	Polska

Zadanie dla studenta:

Utwórz dwa DataFrame'y: `auta1` z kolumnami 'Marka' i 'Model' (`np.DataFrame({'Marka': ['Ford'], 'Model': ['Focus']})`) oraz `auta2` z tymi samymi kolumnami (`np.DataFrame({'Marka': ['Opel'], 'Model': ['Astra']})`). Następnie połącz je w jeden DataFrame wszystkie_auta wierszami.

Zaawansowane operacje i uczenie maszynowe

W uczeniu maszynowym, efektywne przygotowanie i przetwarzanie danych jest kluczowe. Poniżej przedstawiamy fundamentalne metody, które pozwalają na manipulację danymi, ich skalowanie, kodowanie oraz finalnie, wykorzystanie modeli do predykcji.

Metoda .apply() - Stosowanie funkcji do kolumny/wiersza

Wyjaśnienie:

Metoda .apply() w Pandas pozwala na aplikowanie funkcji (własnych lub wbudowanych) wzdłuż osi DataFrame (kolumnowo lub wierszowo). Jest niezwykle przydatna do złożonych operacji na danych, które nie są łatwo dostępne w standardowych metodach Pandas, np. do tworzenia nowych cech na podstawie skomplikowanych reguł.

Przykładowy DataFrame INPUT:

```
import pandas as pd

 dane_sprzedazowe = pd.DataFrame({
     'ProduktID': [1, 2, 3, 4, 5],
     'Cena': [15.50, 22.00, 5.75, 45.00, 18.20],
     'Ilosc': [2, 1, 4, 1, 3],
     'Region': ['Północ', 'Południe', 'Wschód', 'Północ', 'Zachód']
 })
 print("Oryginalny DataFrame dane_sprzedazowe:")
 print(dane_sprzedazowe)
```

Rzeczywisty OUTPUT:

```
Oryginalny DataFrame dane_sprzedazowe:
 ProduktID  Cena  Ilosc  Region
0         1 15.50    2  Północ
1         2 22.00    1 Południe
2         3  5.75    4  Wschód
3         4 45.00    1  Północ
4         5 18.20    3  Zachód
```

Użycie .apply():

```
# Użycie .apply() do obliczenia łącznej wartości sprzedaży z rabatem
def oblicz_wartosc_z_rabatem(row):
    wartosc_bazowa = row['Cena'] * row['Ilosc']
    if wartosc_bazowa > 50:
        return wartosc_bazowa * 0.9 # 10% rabatu
    return wartosc_bazowa

 dane_sprzedazowe['Wartosc_Laczna_Z_Rabat'] = dane_sprzedazowe.apply(oblicz_wartosc_z_rabatem, axis=1)
 print("\nDataFrame z nową kolumną 'Wartosc_Laczna_Z_Rabat':")
 print(dane_sprzedazowe)

# Użycie .apply() z lambda do skategoryzowania cen
 dane_sprzedazowe['Kategoria_Cenowa'] = dane_sprzedazowe['Cena'].apply(lambda x: 'Wysoka' if x > 20 else 'Niska')
 print("\nDataFrame z nową kolumną 'Kategoria_Cenowa':")
 print(dane_sprzedazowe)
```

Rzeczywisty OUTPUT:

```
DataFrame z nową kolumną 'Wartosc_Laczna_Z_Rabat':
 ProduktID  Cena  Ilosc  Region  Wartosc_Laczna_Z_Rabat
0         1 15.50    2  Północ             31.00
1         2 22.00    1 Południe             22.00
2         3  5.75    4  Wschód             23.00
3         4 45.00    1  Północ             45.00
4         5 18.20    3  Zachód             48.60

DataFrame z nową kolumną 'Kategoria_Cenowa':
 ProduktID  Cena  Ilosc  Region  Wartosc_Laczna_Z_Rabat  Kategoria_Cenowa
0         1 15.50    2  Północ             31.00             Niska
1         2 22.00    1 Południe             22.00             Wysoka
2         3  5.75    4  Wschód             23.00             Niska
3         4 45.00    1  Północ             45.00             Wysoka
4         5 18.20    3  Zachód             48.60             Niska
```

Zadanie dla studenta:

Dla DataFrame dane_sprzedazowe, utwórz nową kolumnę 'Duze_Zamowienie', która będzie miała wartość True, jeśli 'Ilosc' jest większa niż 2, w przeciwnym razie False. Użyj metody .apply() z funkcją lambda.

Łączenie DataFramów - concat

W analizie danych i uczeniu maszynowym rzadko zdarza się, aby wszystkie potrzebne informacje znajdowały się w jednym pliku lub jednej tabeli. Często dane są rozproszone i wymagają połączenia z różnych źródeł. Pandas oferuje potężne narzędzia do łączenia obiektów DataFrame, takie jak `pd.concat()`, `.merge()` oraz `.join()`, które pozwalają na efektywne agregowanie danych w zależności od potrzeb. Zrozumienie tych metod jest kluczowe dla sprawnego przygotowywania danych do dalszej obróbki i modelowania.

1. Łączenie DataFramów za pomocą `pd.concat()`

`pd.concat()` służy do łączenia obiektów Pandas (Series lub DataFrame) wzdłuż określonej osi. Można jej używać do dodawania wierszy (łączenie w pionie) lub dodawania kolumn (łączenie w poziomie). Jest to szczególnie przydatne, gdy masz dane o tej samej strukturze, które chcesz zebrać w jedną całość, na przykład dane z różnych okresów czasowych lub segmentów.

Przykładowe DataFrame INPUT:

```
import pandas as pd

df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
df3 = pd.DataFrame({'C': [9, 10], 'D': [11, 12]})

print("DataFrame 1:")
print(df1)
print("\nDataFrame 2:")
print(df2)
print("\nDataFrame 3:")
print(df3)
```

Rzeczywisty OUTPUT:

```
DataFrame 1:
  A  B
0  1  3
1  2  4

DataFrame 2:
  A  B
0  5  7
1  6  8

DataFrame 3:
  C  D
0  9 11
1 10 12
```

Użycie `pd.concat()`:

```
# Łączenie w pionie (domyślnie, axis=0)
concat_rows = pd.concat([df1, df2])
print("\nŁączenie wierszy (axis=0):")
print(concat_rows)

# Łączenie w poziomie (axis=1)
concat_cols = pd.concat([df1, df3], axis=1)
print("\nŁączenie kolumn (axis=1):")
print(concat_cols)
```

Rzeczywisty OUTPUT:

```
Łączenie wierszy (axis=0):
  A  B
0  1  3
1  2  4
0  5  7
1  6  8

Łączenie kolumn (axis=1):
  A  B  C  D
0  1  3  9 11
1  2  4 10 12
```

Zadanie dla studenta:

Na podstawie `df_pracownicy` (kolumny: 'ID', 'Imię', 'Stanowisko') i `df_nowi_pracownicy` (kolumny: 'ID', 'Imię', 'Stanowisko'). Połącz je wierszami, ignorując indeksy, tak aby powstał jeden DataFrame z wszystkimi pracownikami.

Przykładowe DataFrame INPUT:

```
df_pracownicy = pd.DataFrame({
    'ID': [1, 2, 3, 4],
    'Imię': ['Anna', 'Piotr', 'Kasia', 'Jan'],
    'Stanowisko': ['Manager', 'Developer', 'Analitik', 'Tester']
})

df_nowi_pracownicy = pd.DataFrame({
    'ID': [5, 6, 7, 8],
    'Imię': ['Ewa', 'Marek', 'Zofia', 'Adam'],
    'Stanowisko': ['Developer', 'Analitik', 'Manager', 'Tester']
})

print("df_pracownicy:\n", df_pracownicy)
print("\ndf_nowi_pracownicy:\n", df_nowi_pracownicy)

df_pracownicy:
   ID  Imię  Stanowisko
0   1  Anna   Manager
1   2  Piotr  Developer
2   3  Kasia   Analitik
3   4   Jan    Tester

df_nowi_pracownicy:
   ID  Imię  Stanowisko
0   5   Ewa   Developer
1   6  Marek   Analitik
2   7  Zofia   Manager
3   8   Adam    Tester
```

Rzeczywisty OUTPUT:

Jedna tabela o 3 kolumnach i 8 wierszach, ze wszystkimi pracownikami.

Transformacja danych - apply, map

Transformacja danych to kluczowy etap w procesie analizy, pozwalający na dostosowanie struktury i zawartości zbioru danych do konkretnych wymagań. Pandas oferuje wiele potężnych narzędzi do modyfikacji danych, a wśród nich metody `.apply()`, `.map()` wyróżniają się wszechstronnością i efektywnością. Rozumienie ich zastosowań jest bardzo przydatne do sprawnego przygotowania danych do dalszej analizy i modelowania.

1. Transformacje z .apply()

Metoda `.apply()` jest niezwykle elastyczna i pozwala na stosowanie dowolnej funkcji (wbudowanej, niestandardowej lub funkcji lambda) do każdego elementu w kolumnie (Series) lub do każdego wiersza/kolumny w całym DataFrame. Jest to idealne rozwiązanie, gdy potrzebujemy wykonać złożone operacje, które nie są dostępne jako wbudowane metody Pandas.

Przykładowy DataFrame INPUT:

```
import pandas as pd

df_uczniowie = pd.DataFrame({
    'Imię': ['Anna', 'Piotr', 'Kasia', 'Marek', 'Zofia'],
    'Wynik_Testu': [85, 92, 78, 65, 90],
    'Obecność_dni': [180, 175, 160, 150, 178]
})

print("DataFrame Uczniowie:")
print(df_uczniowie)
```

Rzeczywisty OUTPUT:

DataFrame Uczniowie:			
	Imię	Wynik_Testu	Obecność_dni
0	Anna	85	180
1	Piotr	92	175
2	Kasia	78	160
3	Marek	65	150
4	Zofia	90	178

Użycie `.apply()`:

Przykład zastosowania funkcji lambda do stworzenia kolumny 'Status_Wyniku' na podstawie 'Wynik_Testu'.

```
df_uczniowie['Status_Wyniku'] =
df_uczniowie['Wynik_Testu'].apply(
    lambda x: 'Zaliczone' if x >= 70 else 'Do Poprawy'
)

print("\nDataFrame z kolumną 'Status_Wyniku':")
print(df_uczniowie)
```

Rzeczywisty OUTPUT:

DataFrame z kolumną 'Status_Wyniku':				
	Imię	Wynik_Testu	Obecność_dni	Status_Wyniku
0	Anna	85	180	Zaliczone
1	Piotr	92	175	Zaliczone
2	Kasia	78	160	Zaliczone
3	Marek	65	150	Do Poprawy
4	Zofia	90	178	Zaliczone

Zadanie dla studenta:

Stwórz nową kolumnę 'Procent_Obecności' w `df_uczniowie`, obliczając procent obecności każdego ucznia (`Obecność_dni` / 180 dni w roku szkolnym) i zaokrąglając wynik do dwóch miejsc po przecinku, używając metody `.apply()` oraz funkcji `lambda`.

2. Mapowanie wartości za pomocą .map()

Metoda `.map()` służy do mapowania wartości w Series (czyli pojedynczej kolumnie) na inne wartości, używając słownika lub innej Series jako mapy. Jest to szczególnie przydatne do zamiany wartości tekstowych na liczbowe (lub odwrotnie) albo do standaryzacji kategorii.

Przykładowy DataFrame INPUT:

```
df_oceny_opisowe = pd.DataFrame({
    'Przedmiot': ['Matematyka', 'Historia', 'Angielski',
                  'Matematyka', 'Historia'],
    'Ocena_Literowa': ['A', 'B', 'C', 'A', 'A']
})

print("DataFrame Oceny Opisowe:")
print(df_oceny_opisowe)
```

Rzeczywisty OUTPUT:

DataFrame Oceny Opisowe:		
	Przedmiot	Ocena_Literowa
0	Matematyka	A
1	Historia	B
2	Angielski	C
3	Matematyka	A
4	Historia	A

Użycie `.map()`:

Mapowanie ocen literowych na odpowiadające im wartości liczbowe.

```
mapa_ocen = {'A': 5, 'B': 4, 'C': 3, 'D': 2, 'F': 1}
df_oceny_opisowe['Ocena_Liczbowa'] =
df_oceny_opisowe['Ocena_Literowa'].map(mapa_ocen)

print("\nDataFrame z kolumną 'Ocena_Liczbowa':")
print(df_oceny_opisowe)
```

Rzeczywisty OUTPUT:

DataFrame z kolumną 'Ocena_Liczbowa':			
	Przedmiot	Ocena_Literowa	Ocena_Liczbowa
0	Matematyka	A	5
1	Historia	B	4
2	Angielski	C	3
3	Matematyka	A	5
4	Historia	A	5

Zadanie dla studenta:

Mając DataFrame `df_oceny_opisowe`, stwórz nową kolumnę 'Poziom_Trudności' dla przedmiotu, mapując 'Matematyka' na 'Wysoki', 'Historia' na 'Średni', a 'Angielski' na 'Niski'.

